

Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 8

Document Revision History

Version	Date	Author	Comments
1.0	09/17/2026	Megan Wasner	Created document and created and updated all data inside.
2.0	10/11/2026	Megan Wasner	Updated evaluation section.
3.0	10/18/2026	Megan Wasner	Updated recommendations section

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Develop a game called "Draw It or Lose it". It will consist of a multiple player game where players will generate stock images to help other players guess the puzzle, which can be a title phrase or thing. There will be multiple teams competing against each other. There will be four rounds that last 30 seconds. If your team fails to guess the puzzle within the allotted time, other teams will have the opportunity to guess and gain your point. To obtain this we must have a web-based application that is able to connect players online. Since there is an android application we must also consider allowing mobile and web users to play the same game using these different devices.

Requirements

The gaming room has designated a few development requirements. Games must consist of teams with multiple players on each time. Each game and team has to have unique identifiers. Only one instance of the game can exist at a time. It must be connected to a stock photo library to allow users to player. It must serve multiple platforms and potentially work alongside their current mobile game.

Design Constraints

1. Connecting web based users and android users to the same application so they can play together
2. Since this is a web application that allows users in different environments to play together, security is a concern. Especially if users have individual accounts. How would we manage protecting user data during play?
3. Since this application has a prior expectation of appearance and performance because of the mobile app we must uphold these expectations. If we do not users could simply choose to use the mobile application instead based on preference.
4. Should users have the option to omit certain topics or images, based on user age or needs?
5. Are there any financial or time constraints?

System Architecture View

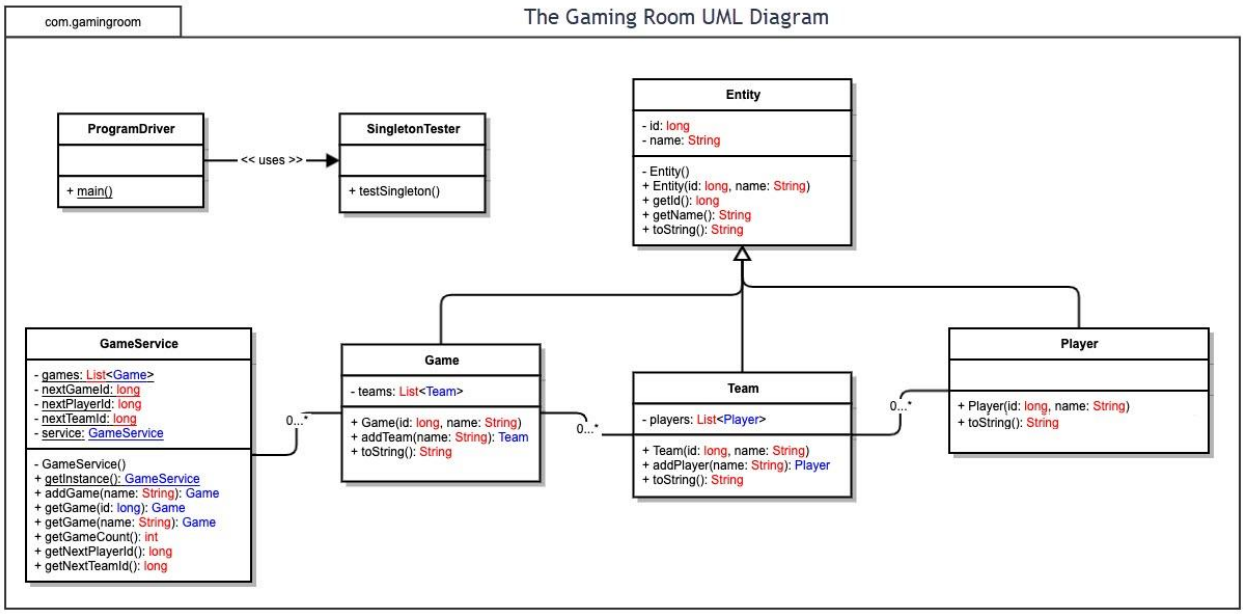
Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The Entity class is the base superclass for the program. The Game, Team, and Player class all extend from the Entity class. Team belongs to Game and contains Player objects, while Game contains a connection of team objects. Player represents the individuals that will make up the team class. Gameservice is a singleton that manages the lifecycle of the games, it also holds the private data for each game player and team. Its singleton pattern is through getInstance(). It also maintains a collection of the active games being played. The program driver holds the main class and connects to the singleton tester which is used for system validation.

OOP principles present are inheritance, indicated by the game, team, and player classes inheriting from the Entity class, this will help prevent redundance. Encapsulation is also present, since there is private attributes paired with public accessor and mutator functions, this hides internal data and prevents

external classes from altering internal data structure, this will reduce potential bugs! And finally there is plenty of polymorphism and abstraction, indicated by common behaviors being inherited from the entity class, and things like the list collection allows abstract details to exist in list interface.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	MacOS is a stable desktop environment, but hosting enterprise webservers is complex after the discontinuation of MacOS Server. So hosting dedicated MacOS servers in the cloud can be complex due to the need for mac-centric rentals or hardware, which can quickly make it financially complex.	Linux is the cheapest and most accessible for hosting web-based applications. It's open source, allows for high performance, and is secure and stable, while allowing zero licensing fees. The downside being the steep learning curve for administration and management of the application, since it requires more manual configuration compared to other platforms.	Windows server has extensive enterprise support and deployment through Azure. It also allows for native active directory integration, GUI based application management. But it has multiple operating system licensing costs, takes quite a bit of hardware resources, and requires frequent patching.	Mobile devices are not suitable for server-side web applications! Its just not sustainable for multiple reasons, like dynamic Ip addresses, lack of enterprise hardware reliability, and there just isn't enough power in the devices.

Client Side	Since this is a web-based application, users can use modern web browsers like chrome or Firefox to access it. Mac offers an easy way to access applications without much integration.	Though the potential client market for Linux users may be small, it will be accessible through standard browsers. It is important to ensure Linux compatibility despite the smaller clientele.	Since windows is undoubtably the largest desktop client base making it the most essential to iron out and support. Again they are minimal extra cost and it's extremely accessible through applications like chrome and Firefox.	If clients use the web-based application on mobile devices it Is essential to create a dedicated mobile web version of the application, that way it still can use touch interface and have proper scaling. It also would be important to state to mobile web users that there is an existing mobile application. Its also important to implement further testing due to the addition of new desktop applications. Using software like Playwright is essential to make sure that the application looks and works the same across different devices.

Development Tools	Essential development tools include things like Swift, or Apple Xcode IDE. For specifically web development things like Node.js, JetBrains Webstorm, or Docker, all run natively on mac and are good options. Licensing costs is something to be considered however, as apple has a 99 dollar a year licensing fee for ecosystem distribution. For a development team only one team is needed across operating systems due to the web-based development.	Linux relies on heavy terminal environments like Bash, and then lighter coding editors like Jetbrains IDE or Vim. Jetbrains has around a 19 dollar a month licensing fee per individual! For a development team only one team is needed across operating systems due to the web-based development.	Windows development is really centered around Visual Studio. Frameworks like .NET, and tools like docker desktop make windows really versatile for enterprise and web software engineering! The only tool here that could cost some money is using Visual Studio professional which costs around 45 dollars a month per individual For a development team only one team is needed across operating systems due to the web-based development.	Web applications for mobile devices require frameworks like React or Vue to be created specifically for mobile. Tools like Xcode, or Android studio are also important for mobile device emulation. Framework tools like flutter are needed for debugging as well. Most of the tools used for mobile development are free! The only main cost being the licensing costs related to the apple mobile ecosystem which is 99 dollars a year. For a development team only one team is needed across operating systems due to the web-based development.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: The best choice for operating platform for this application is by far Linux (Ubuntu or Debian). Not only is it the industry standard, but it minimizes cost, extremely reliable, and is overwhelmingly scalable. Through modern web interfaces users can connect seamlessly across devices through web browsers, making Linux the most reasonable choice.

2. **Operating Systems Architectures**: The server will run with containers through an application like Docker through Linux kernel and allows users to have their own game environment by using kernel features like namespaces. Using docker aligns with having Linux as our operating platform and allows for secure isolated sessions between players!

3. **Storage Management**: A hybrid approach to storage architecture is the best choice. Having a relational database for storage like user profiles, game history. And an in-memory cache to store active rooms, turns, timers. Is the most effective and sustainable approach!

4. **Memory Management**: Linux uses a robust virtual memory management system; it allows for automated memory for active game team and player instances. As games end and players leave this data is reallocated where needed. This ensures optimal utilization and prevents memory leaks across concurrent sessions

5. **Distributed Systems and Networks**: to support a real time multiplayer application like a client server architecture through something like WebSocket's is the best choice. It also would need a load balancer like NGINX to help prevent connection drops, server outages, or network latency. and route them to different places in the system.

6. **Security**: Security is extremely important. So all communication between servers would have to be encrypted using TLS 1.3. It would also benefit from web authorization using Web Tokens to verify WebSocket requests. Passwords would need to be hashed before storage and prevention of unauthorized access through firewalls would be needed as well.